UNIVERSITATEA TEHNICĂ A MOLDOVEI

Технический университет Молдовы

Факультет вычислительной техники,
информатики и микроэлектроники

Департамент программной инженерии и автоматики

Программирование мобильных приложений

Programarea aplicațiilor mobile

Отчёт по лабораторной работе № 4

Состояние и репозитории
Индивидуальная тема RunBaza

Выполнил: Hamuev Oleg

Группа: CR-232

Проверил: Leah A.

Учебный год 2026–2027

Кишинёв 2026

1 Цель и результат работы

В Л4 приложение RunBaza получает общее состояние и асинхронный слой данных. Формы участия, поездок и профиля сохраняют изменения через репозитории. После успешной операции детали забега, личный план, счётчик вкладки и статистика профиля обновляются через ChangeNotifier.

Тема проекта RunBaza: календарь забегов и поиск попутчиков в Молдове и Румынии. Продолжается проект Lab2/run_baza, созданный в Л2 и дополненный навигацией и валидацией в Л3. Пять вкладок, маршруты по ID и анимированные иллюстрации дистанций сохранены.

Соответствие заданию

Требование	Реализация
Общее состояние	RaceCatalog, PlanModel, OfferModel, RequestModel, ProfileModel
Асинхронный CRUD	Типизированные Repository и sealed Result с Ok и Err
Mock источник	Память, уникальные ID, задержка 800 мс, управляемая ошибка
Внедрение зависимостей	Выбор реализации и Provider в main.dart; интерфейс в конструкторе модели
Три состояния	LoadingView, содержимое, ErrorView с действием «Повторить»
Изменения между экранами	Добавление, редактирование и удаление участия; общие счётчики
Связанные формы	Участие, водитель, пассажир и профиль пишут через модели
Чистота UI	Seed импортируется только в mock_repositories.dart
Проверка	125 unit/widget-тестов и сценарий на iPhone Simulator

Границы реализации

Mock хранит данные в памяти текущего процесса. После полного перезапуска приложения восстанавливается начальный набор. Реальный HTTP, постоянная база, серверная авторизация, бронирование и доставка уведомлений в Л4 не реализованы. Вход и регистрация остаются локальной проверкой полей.

2 Архитектура приложения

Экран получает готовое состояние модели и вызывает её действия. Модель зависит от интерфейса хранилища. Конкретная реализация выбирается в main.dart, поэтому источник можно заменить без переписывания экранов.

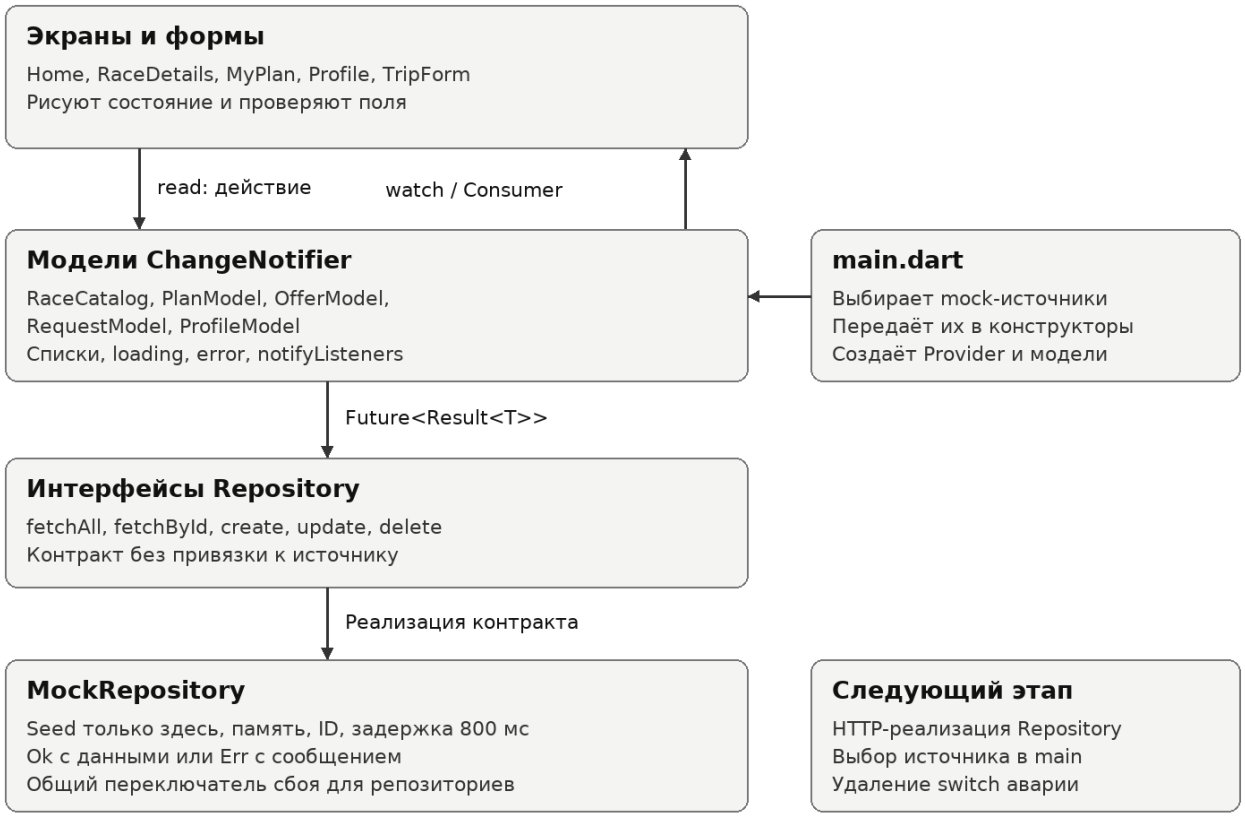


Рисунок 1. Схема слоёв RunBaza и направление зависимостей.

Владение состоянием

Сохранённые записи и настройки доступны нескольким экранам, поэтому живут в моделях над MaterialApp.router. Поиск, выбранный фильтр, незавершённые поля формы и раскрытие раздела диагностики остаются в State конкретного виджета.

Переход к сетевому источнику

На следующем этапе добавляется HTTP-реализация тех же интерфейсов и меняется выбор источника в main.dart. Из профиля убирается переключатель mock-ошибки. Контракт Result должен сохраняться: ожидаемые ошибки запроса преобразуются в Err внутри репозитория. Экран и модель не получают HTTP-клиент.

3 Контракт и mock репозиторий

Общий Repository<T> задаёт пять операций для сущностей с ID. Отдельные интерфейсы RaceRepository, PlanRepository, OfferRepository, RequestRepository и ProfileRepository фиксируют тип данных. Модели не создают себе mock.

```
import 'entity.dart';
import 'result.dart';

abstract interface class Repository<T extends Entity> {
  Future<Result<List<T>>> fetchAll();
  Future<Result<T>> fetchById(String id);
  Future<Result<T>> create(T draft);
  Future<Result<T>> update(T record);
  Future<Result<void>> delete(String id);
}

sealed class Result<T> {
  const Result();
}

final class Ok<T> extends Result<T> {
  const Ok(this.value);
  final T value;
}

final class Err<T> extends Result<T> {
  const Err(this.msg);
  final String msg;
}
```

Чтение из памяти с задержкой

```
  Future<bool> _fails() async {
    await Future<void>.delayed(latency);
    return failure.enabled;
  }

  static const _networkError =
      'Не удалось выполнить запрос. Попробуйте ещё раз.';
  static const _missing = 'Запись больше не существует.';

  @override
  Future<Result<List<T>>> fetchAll() async {
    if (await _fails()) return const Err(_networkError);
    return Ok(List.unmodifiable(_records));
  }
```

Все CRUD-операции используют ту же задержку и флаг ошибки. При Err данные не меняются. create выдаёт новый ID с префиксом и проверкой коллизий; update и delete проверяют существование записи. Возвращаемый список защищён от изменения извне.

4 Загрузка и изменение состояния

CollectionModel хранит приватный список, флаги загрузки и сохранения, текст ошибки. Наследники задают тип репозитория и действия предметной области. Номер запроса защищает от запоздалого ответа предыдущей загрузки.

```
  Future<void> load() async {
    if (_disposed || _saving) return;
    final request = ++_revision;
    _loading = true;
    _error = null;
    notifyListeners();
    final result = await _repository.fetchAll();
    if (_disposed || request != _revision) return;
    switch (result) {
      case Ok(:final value):
        _records = List.of(value);
      case Err(:final msg):
        _error = msg;
    }
    _loading = false;
    notifyListeners();
  }

List<T> get records => List.unmodifiable(_records);

Future<String?> add(T draft) => _write(
  () => _repository.create(draft),
  (created) => _records = [..._records, created],
);
```

load уведомляет подписчиков до ожидания и после результата. Общий _write блокирует повторную запись, ждёт repository, применяет только Ok и вызывает notifyListeners. При ошибке возвращается сообщение, при успехе null. После dispose уведомления не отправляются.

Одна запись участия на забег

PlanModel.saveForRace ищет запись по raceId. Если она существует, выполняется update с тем же ID; иначе create. Изменение дистанции не создаёт второй одинаковый забег в плане. Dismissible использует ValueKey(entry.id), поэтому состояние строки связано с записью.

5 Внедрение зависимостей и подписки

RunBazaScope в main.dart создаёт пять репозиториев и общий MockFailure один раз на время жизни приложения. Обычный Provider передаёт интерфейс, ChangeNotifierProvider создаёт и освобождает модель. lazy: false запускает загрузку сразу.

```
final _failure = MockFailure();
late final _plan = MockPlanRepository(
  _failure, latency: widget.latency,
);

Provider<PlanRepository>.value(value: _plan),
ChangeNotifierProvider(
  lazy: false,
  create: (context) =>
      PlanModel(context.read<PlanRepository>())..load(),
),
```

Чтение для экрана и чтение для действия

```
final plan = context.watch<PlanModel>();
final catalog = context.watch<RaceCatalog>();

// In an action callback:
final error = await context.read<PlanModel>().remove(id);
if (!context.mounted) return;

// Only the badge subtree needs this subscription:
Consumer<PlanModel>(
  builder: (context, plan, child) => Badge(
    label: Text("${plan.count}"),
    isLabelVisible: plan.count > 0,
    child: child,
  ),
  child: const Icon(Icons.bookmark_border),
)
```

watch устанавливает подписку в build. read получает объект для однократного действия и не подписывает callback. Consumer ограничивает перестроение счётчиком. Provider для mock в профиле используется только переключателем диагностики, как предусмотрено заданием.

Отсутствующая запись

Детали ищут сущность через model.byId(id). Пока идёт запрос, показывается загрузка. После завершения неизвестный ID приводит на экран «Страница не найдена», а не к бесконечному индикатору или исключению firstWhere.

6 Загрузка и данные главного экрана

Главная подписана на каталог и предложения поездок. Задержка mock делает ожидание видимым. После загрузки количество событий и свободных мест вычисляется из моделей.

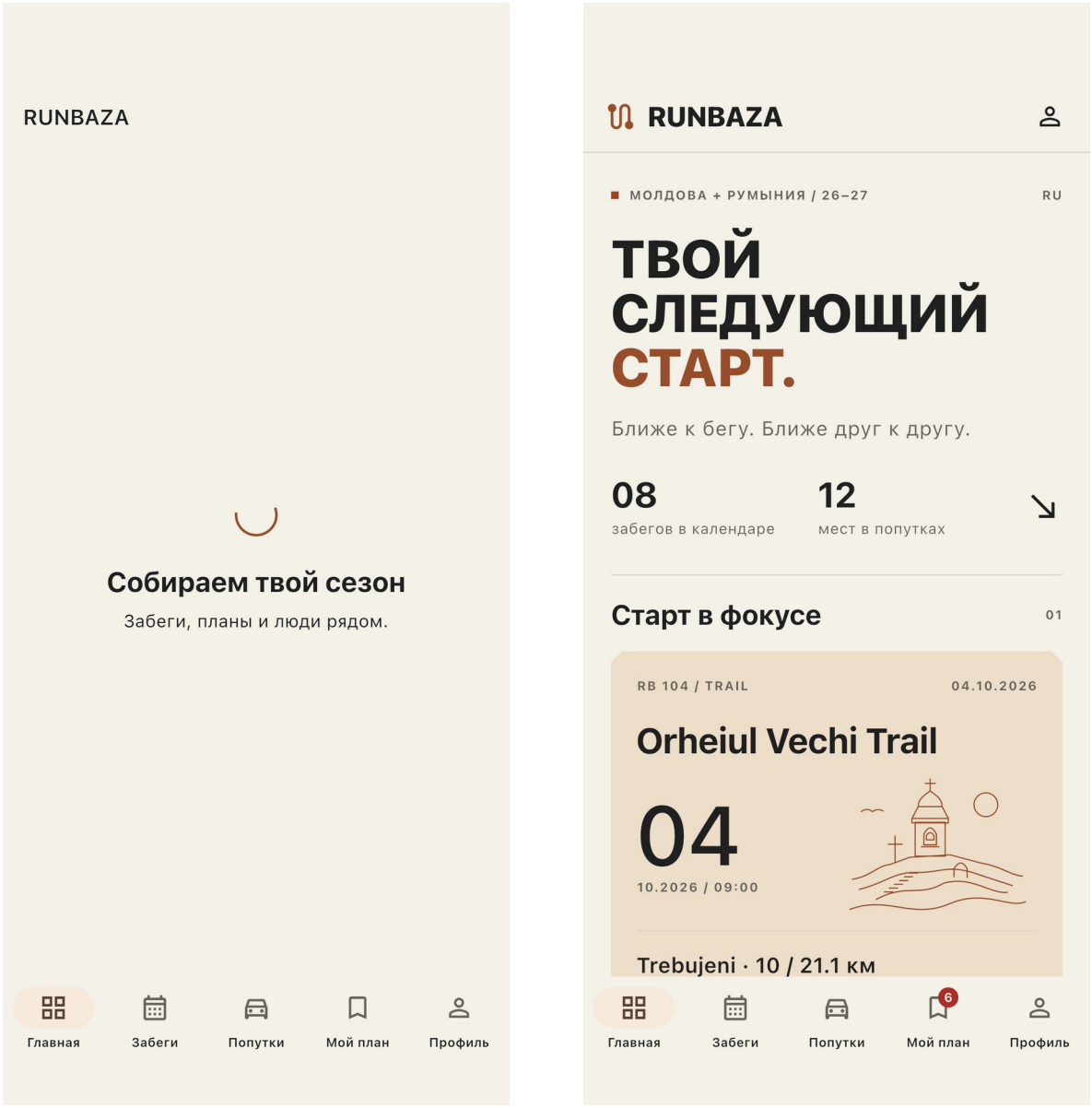


Рисунок 2. Один экран в состояниях loading и data.

Снимки получены из integration_test на iPhone 17 Pro Simulator с iOS 26.5. Первое состояние снято до окончания начального запроса; второе после завершения. Подобная проверка состояния применяется и к остальным экранам с данными.

7 Ошибка и повторная загрузка

В профиле раздел «Диагностика» содержит «Симулировать сбой сети». После включения и обновления каталога repository возвращает Err. Главная показывает сообщение и кнопку повторной попытки.

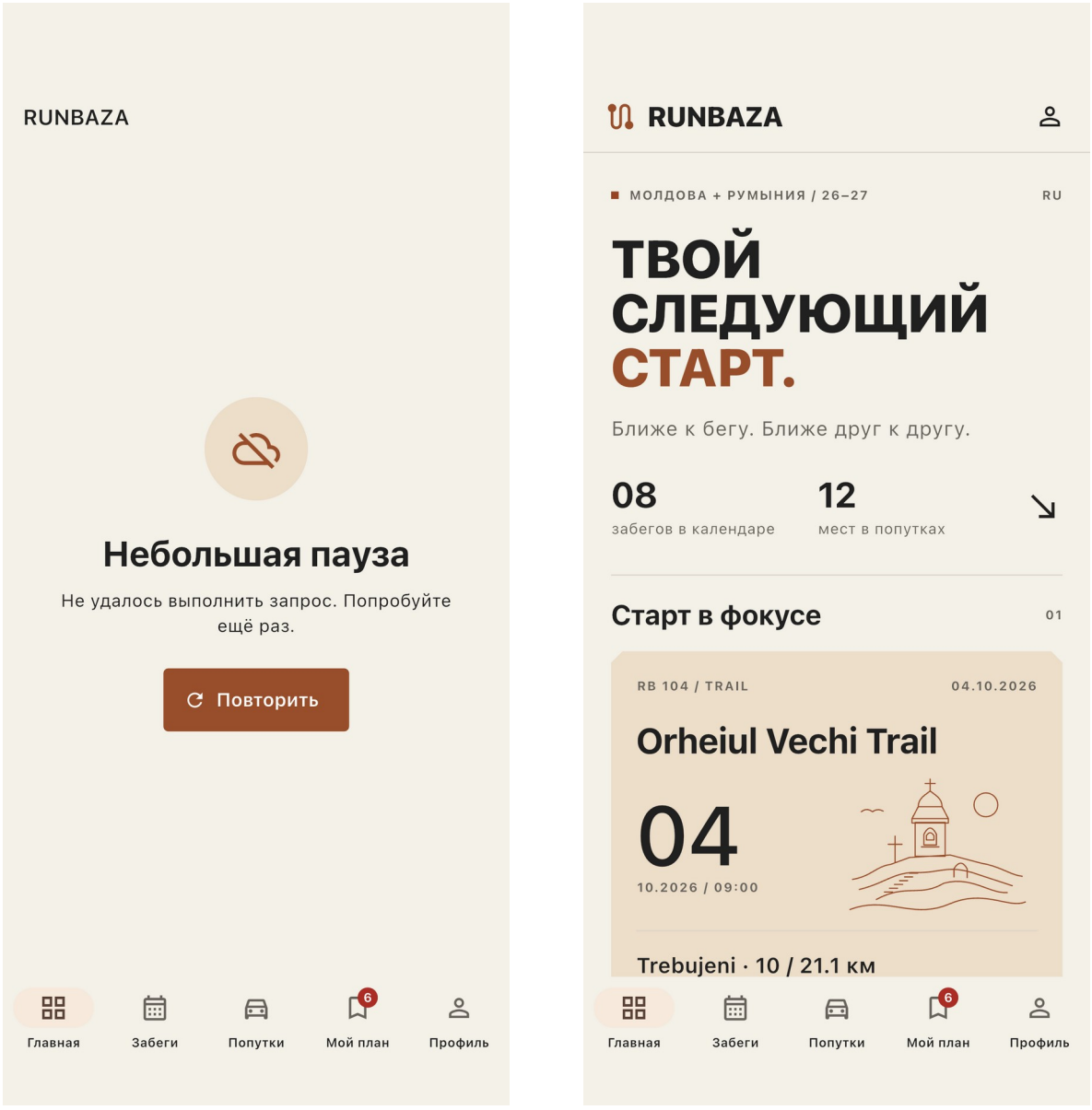


Рисунок 3. Ошибка чтения и данные после отключения сбоя и retry.

Повторная попытка при включённом сбое снова завершается ошибкой без падения приложения. После выключения флага тот же load возвращает данные. Переключатель доступен даже на ошибочном экране профиля, чтобы можно было восстановить работу.

8 Удаление из плана и общий счётчик

В исходном плане шесть стартов, включая Cricova Wine Cellar Run на 5 км. Подтверждение удаления вызывает PlanModel.remove. Удаляется участие пользователя; сам забег остаётся в общем каталоге.

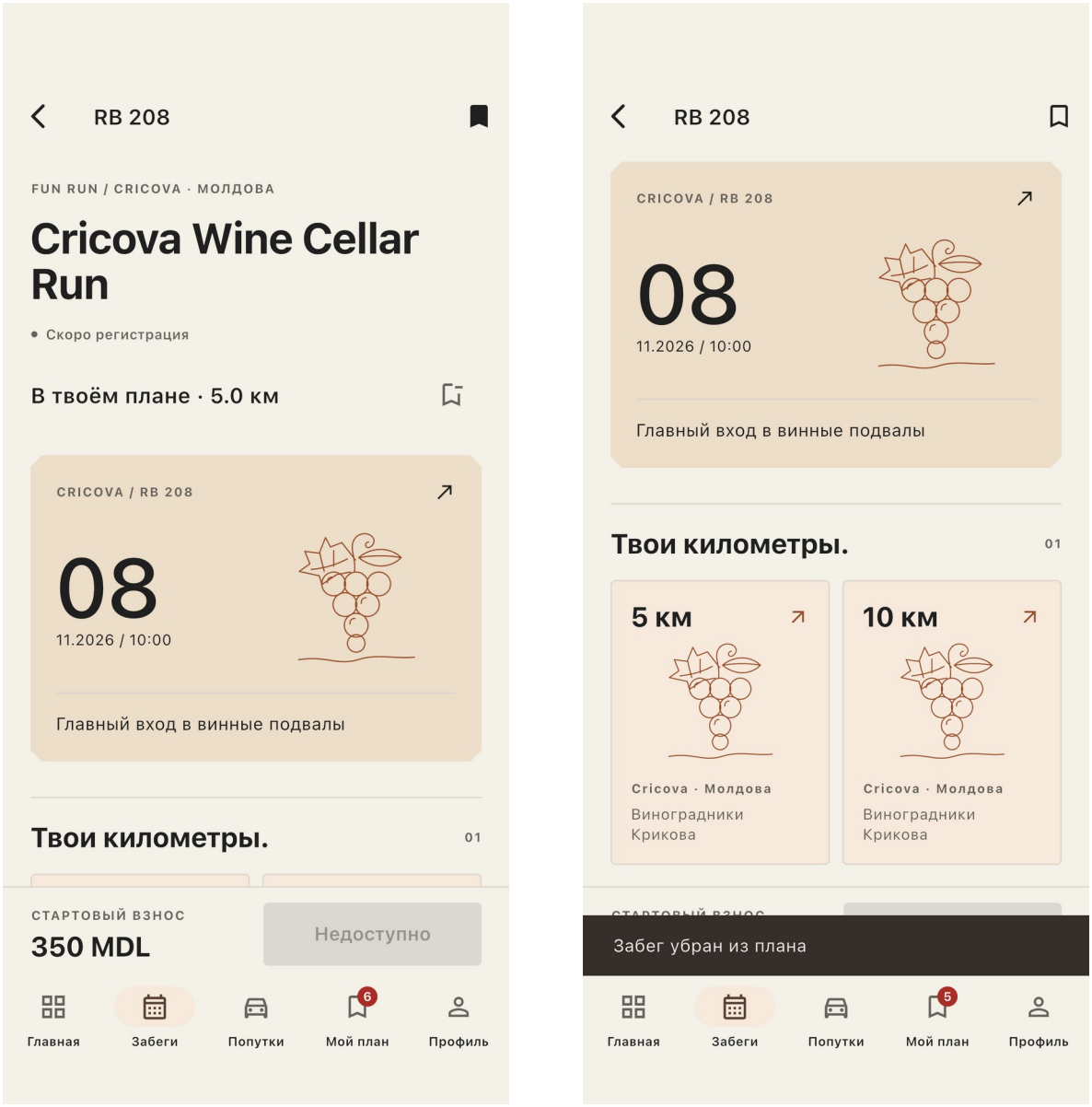


Рисунок 4. Детали и badge до удаления и после него: 6 и 5 записей.

Маркер сохранённого старта исчезает в деталях. Счётчик вкладки получает новое значение через Consumer. Раздел «Мой план» и статистика профиля используют тот же экземпляр PlanModel, поэтому не требуют ручной перезагрузки.

9 Форма участия и результат в списке

После удаления форма снова добавляет Cricova, теперь на 10 км с темпом 5:30. Сохранение проходит через saveForRace и create. На время запроса кнопка заблокирована, повторный submit не создаёт дубликат.

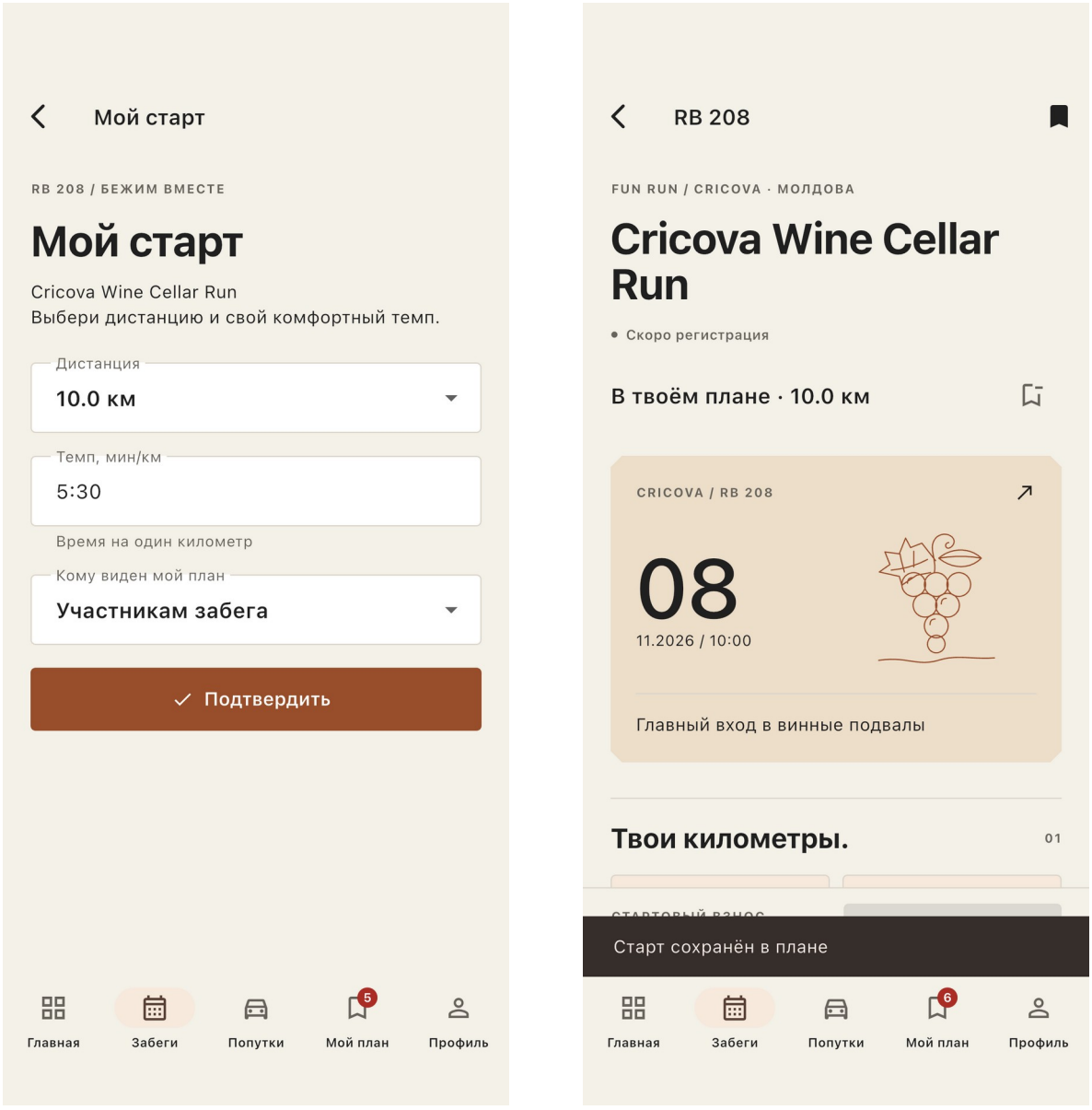


Рисунок 5. Валидная форма с badge 5 и сохранённое участие с badge 6.

Список обновляется до возврата с формы. Настройки дистанции, темпа и видимости находятся в PlanEntry. Видимость является сохранённым параметром плана; публикации другим аккаунтам без backend на этом этапе нет.

10 Согласованность списка и статистики

Один и тот же план после удаления содержит 5 стартов и 86,3 км. После повторного добавления Cricova на 10 км он содержит 6 стартов и 96,3 км. Числа вычислены из сохранённых дистанций записей.

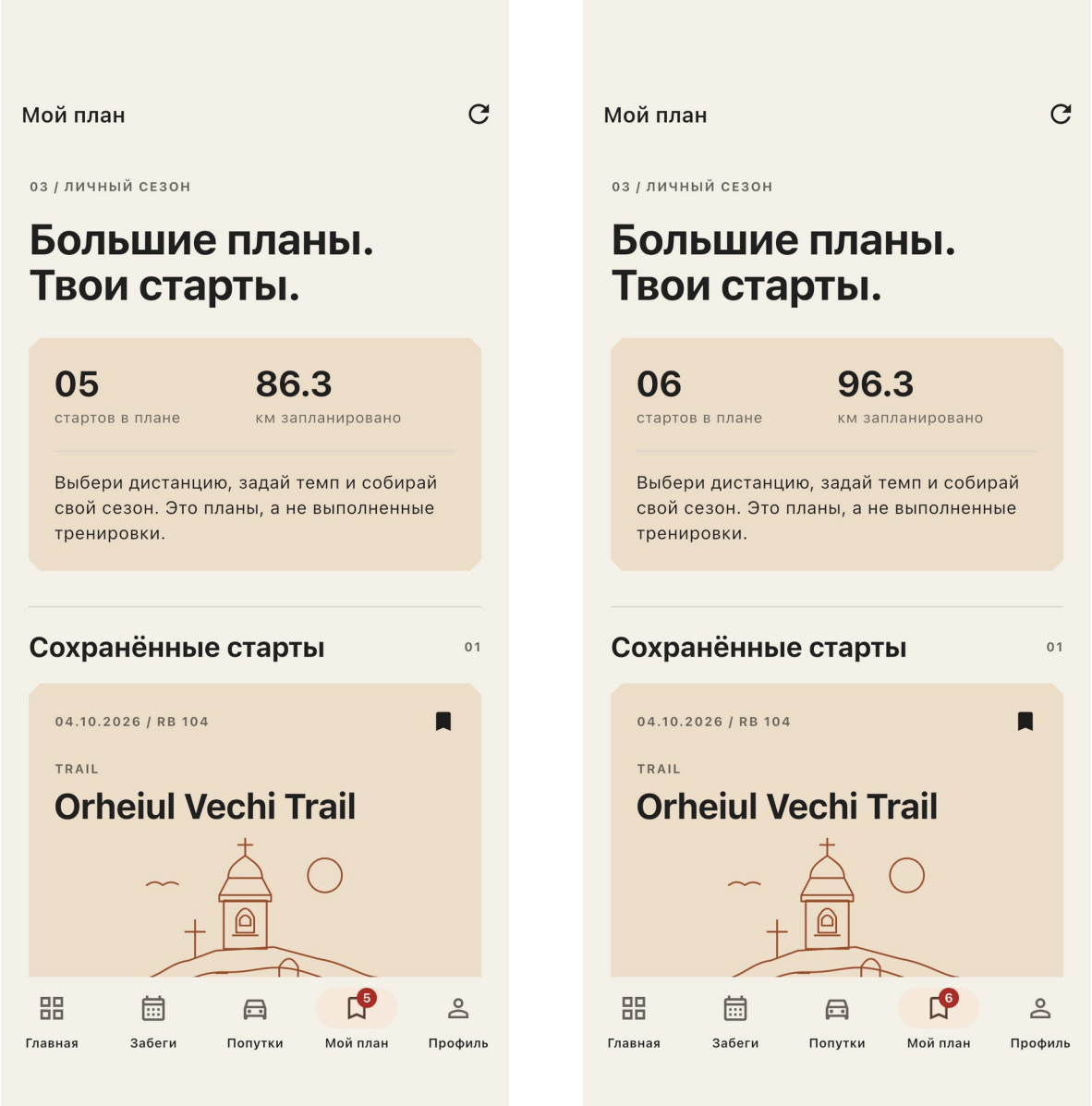


Рисунок 6. Список и суммарная дистанция до добавления и после него.

Если редактируется существующее участие, количество стартов остаётся прежним, а километры пересчитываются. Счётчики описывают запланированные события; приложение не выдаёт их за выполненные тренировки или подтверждённые финиши.

11 Создание поездки и обработка ошибки

Форма водителя проверяет имя, город, место встречи, время, места, вклад и телефон. Затем создаёт TransferOffer через OfferModel. Форма пассажира аналогично создаёт SeatRequest без поля цены.

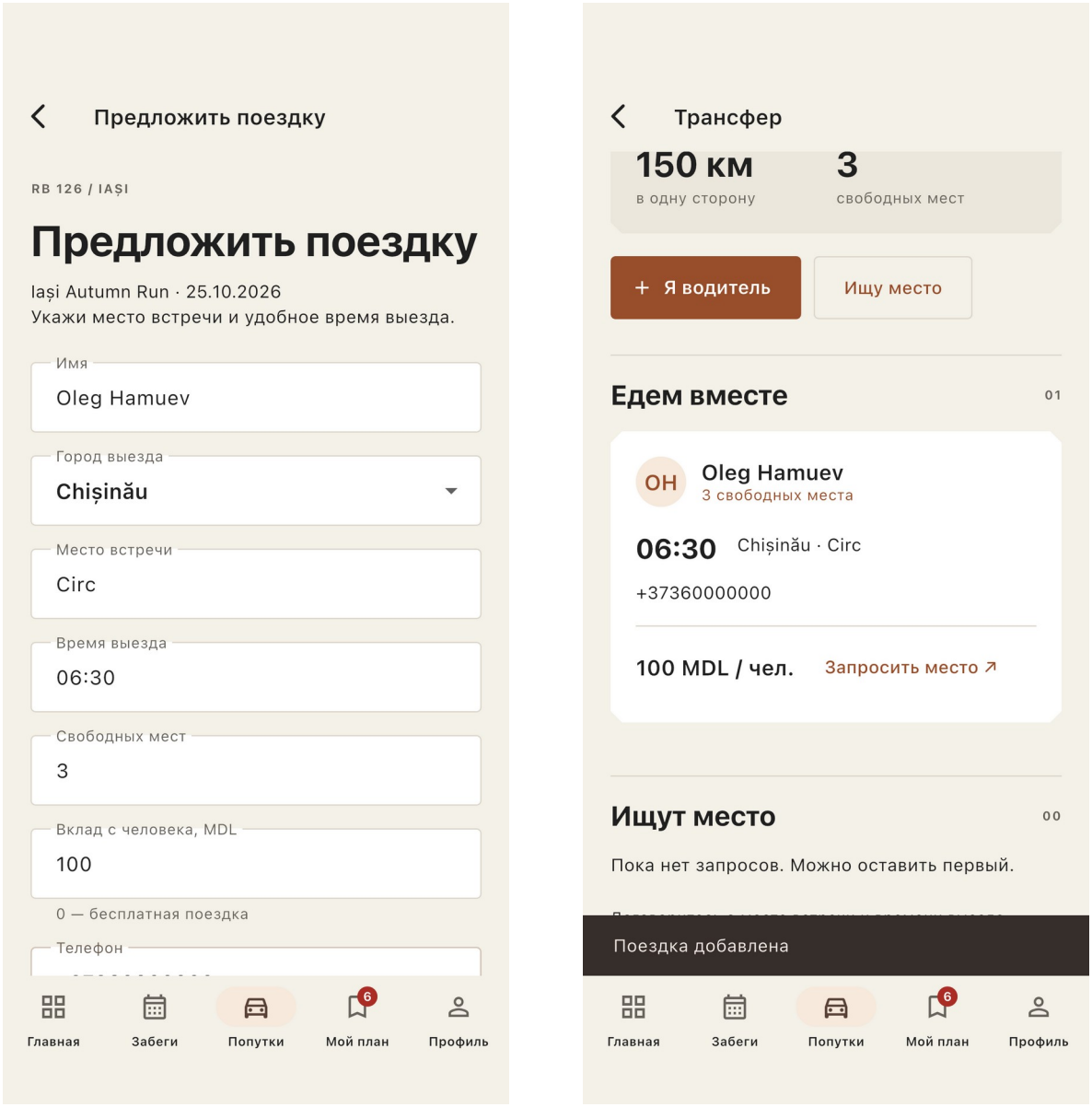


Рисунок 7. Форма водителя и созданное предложение на странице Iași.

В проверке добавлено предложение Oleg Hamuev: выезд из Chișinău, место Circ, 06:30, 3 места, вклад 100 MDL, телефон +37360000000. Репозиторий присваивает ID, а список выбранного забега получает новую карточку.

12 Сохранение полей при сбое

Та же заполненная форма отправлена с включённой аварией. Mock возвращает Err, сообщение появляется в SnackBar, запись не добавляется. Поля остаются заполненными. После отключения аварии повторная отправка создаёт ровно одну запись.

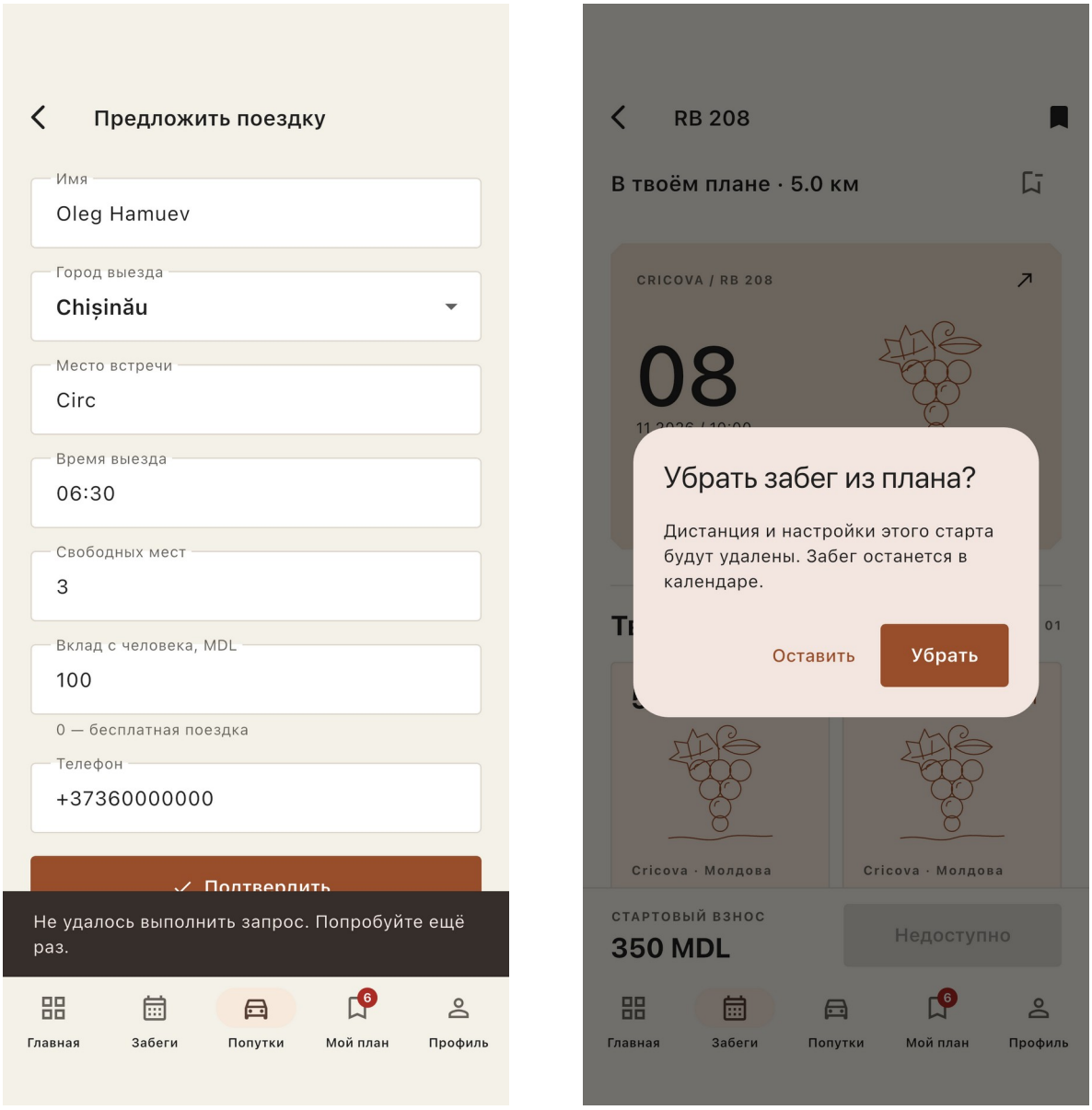


Рисунок 8. Ошибка submit без потери полей и подтверждение удаления.

После await проверяется mounted. Закрытый экран не выполняет переход и не показывает SnackBar. Деструктивное действие выполняется только после true из диалога; отмена оставляет данные без изменений.

13 Новый профиль бегуна

Профиль разделён на обзор и редактирование. Беговой паспорт показывает имя, город, биографию, темп, любимую дистанцию и e-mail. Ниже находятся цель сезона, плановые километры, страны и сохранённые старты.

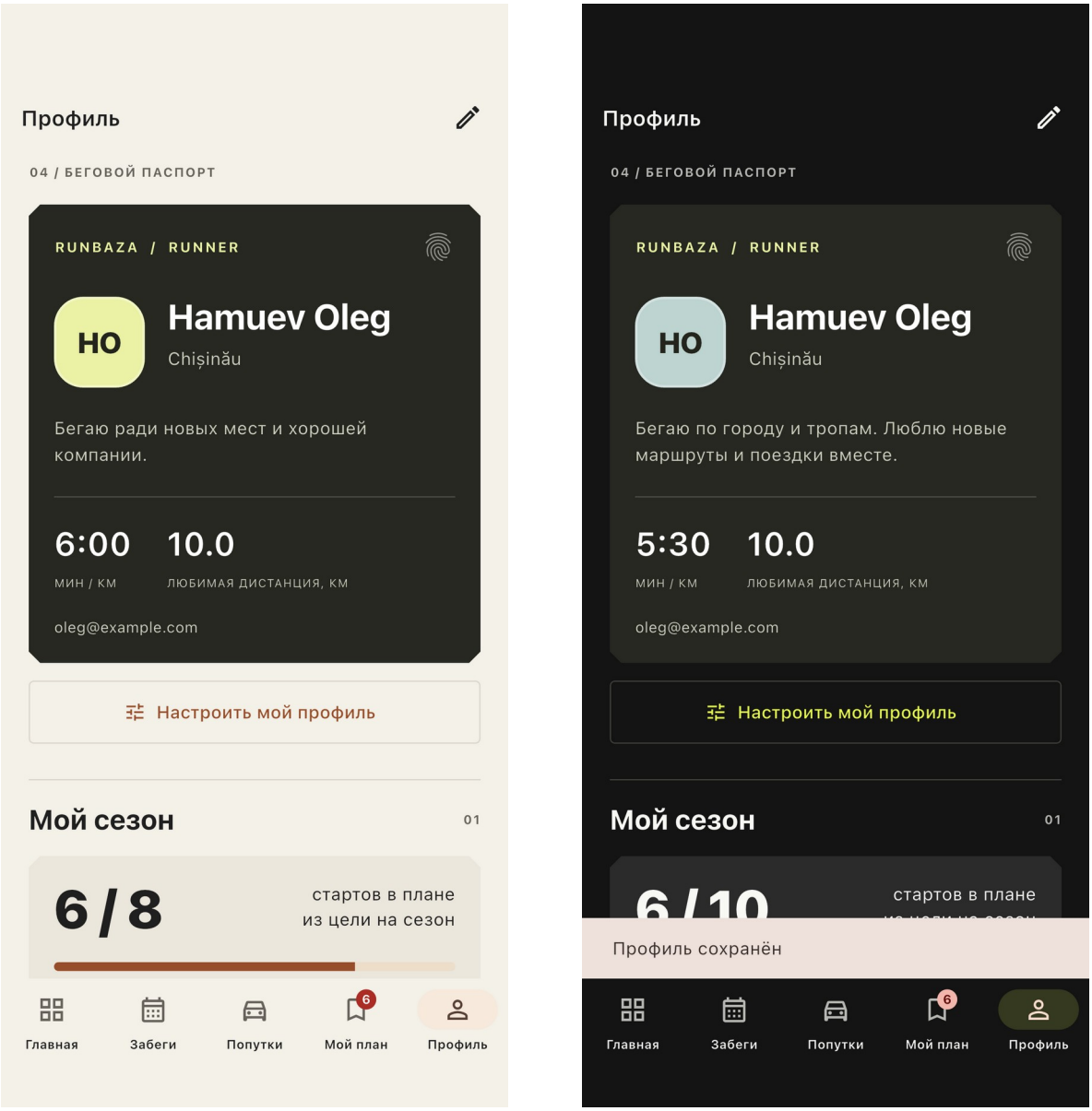


Рисунок 9. Беговой паспорт в светлой теме и после смены настроек в тёмной.

Профиль содержит четыре цвета аватара с инициалами. Выбор «Система», «Светлая» или «Тёмная» сохраняется через ProfileModel и меняет ThemeMode всего приложения. Цель сезона связана с количеством стартов в плане.

14 Редактирование профиля

Отдельный маршрут /profile/edit сохраняет данные и предпочтения через ProfileRepository. Для новых участий используются комфортный темп, доступная любимая дистанция и настройка видимости из профиля.

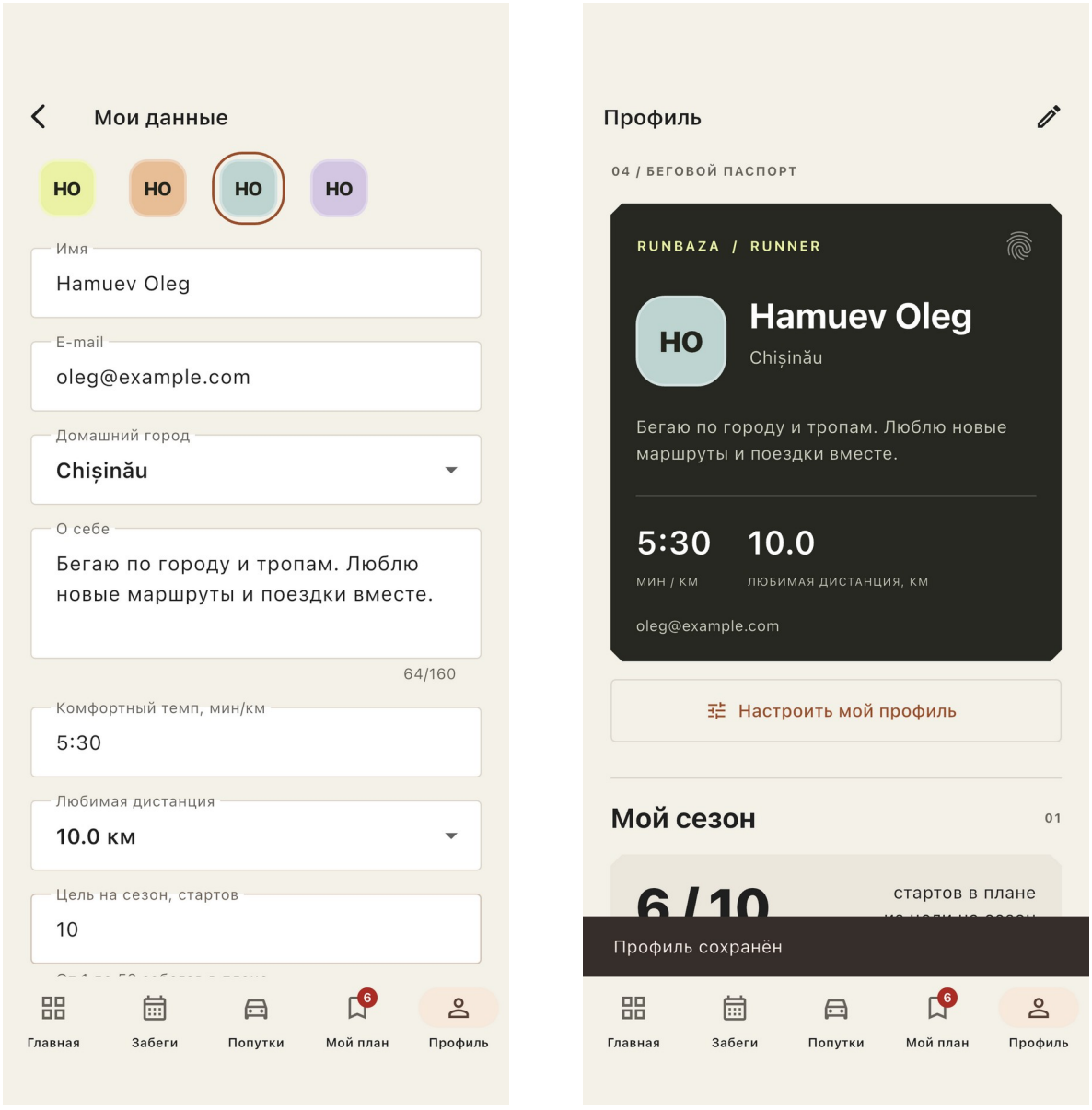


Рисунок 10. Заполненные поля и беговой паспорт после сохранения.

Имя ограничено 2-60 символами, биография 160 символами. E-mail проверяется валидатором, темп допускается от 3:00 до 15:00 мин/км, сезонная цель от 1 до 52 стартов. Ошибка ввода удерживает форму открытой. Ошибка repository сохраняет значения контроллеров.

15 Проверка качества и границы данных

Проверки выполнены 21.10.2026: Flutter 3.47.0, Dart 3.13, iPhone 17 Pro Simulator, iOS 26.5. После интеграционного сценария собрана и установлена обычная версия приложения.

Проверка	Результат
Форматирование и анализ	dart format без изменений; flutter analyze --fatal-infos без замечаний
Unit и widget тесты	125 тестов пройдены
iOS сборка	flutter build ios --simulator --debug --no-codesign завершена успешно
Интеграция	Один сценарий Л4 и служебный teardown; 20 снимков, все проверки пройдены
Контракты данных	CRUD пяти репозиториев, отсутствие ID, уникальные ID, immutable списки, delay
Асинхронное состояние	Уведомления, запоздалые загрузки, повторный submit, завершение после dispose
UI и формы	Обновление нескольких экранов, retry, сохранение полей, тема и валидация
Адаптивность	Ширины 320, 390 и 430; светлая и тёмная темы; увеличенный текст

```
flutter analyze --fatal-infos
flutter test
flutter build ios --simulator --debug --no-codesign
flutter drive --driver=test_driver/lab4_driver.dart \
  --target=integration_test/lab4_test.dart -d <simulator-id>
rg -n "mock_data" lib/screens lib/widgets lib/models
```

Последняя команда не находит совпадений. Seed доступен только mock-репозиториям; сущности вынесены в отдельные файлы. Код не выполняет сетевые запросы. Проверки не доказывают работу будущего backend или сохранение между запусками.

Результат

Формы Л3 стали действиями над общими данными. Архитектура разделяет интерфейс, состояние и источник. Профиль использует реальные изменения текущего сеанса и согласованные показатели плана. Следующий шаг может заменить mock, сохранив контракт моделей и экранов.

16 История Git

Полная последовательность коммитов от Л2 до версии кода отчёта, новые сверху. Ветка lab/4-state-repository. Коммит с итоговыми документами добавляется после формирования этого снимка истории.

```
df5c9da Lab 4: document state ownership and the iOS demonstration
dd4ba6e Lab 4: verify cross-screen updates and error recovery on iOS
5bd2709 Lab 4: reuse profile defaults and show pending plan changes
a9f462b Lab 4: keep seed data inside mock repositories
be87421 Lab 4: update form and layout tests for repository-backed screens
c5a6ab5 Lab 4: redesign the runner profile and save personal preferences
6637182 Lab 4: guard form callbacks after asynchronous writes
22c27b7 Lab 4: create driver offers and passenger requests from forms
4a63ae3 Lab 4: load the transfer directory through the catalog
a2a907a Lab 4: save participation forms through the plan repository
f29076b Lab 4: share plan counts and confirm removal across screens
bcb3eee Lab 4: read distance pages through the catalog model
b8cf8c3 Lab 4: read race details through the catalog model
7747520 Lab 4: add catalog loading, errors and retry
7aea7e2 Lab 4: load the home screen from shared repository state
d2de5c0 Lab 4: provide shared models above the router and add retry views
ae8a5de Lab 4: model shared state with loading and mutation notifications
2ac43da Lab 4: add repository contracts and delayed mock CRUD
22a5300 Lab 4: add provider dependency
03c3974 Use English in lab documentation and code comments
```

16 История Git продолжение

Полная последовательность коммитов от Л2 до версии кода отчёта, новые сверху. Ветка lab/4-state-repository. Коммит с итоговыми документами добавляется после формирования этого снимка истории.

```
cdc6c65 Lab 3: add registration form and session profile
f55bdd8 Lab 3: add Word and PDF reports, defense guide, and iOS screenshots
8f003bc Lab 3: keep all form fields mounted while scrolling
cf74c2a Lab 3: scroll long forms in the iPhone integration scenario
3997ef0 Lab 3: add navigation and form tests, iOS scenarios, and launch guide
4f33fc4 Lab 3: add validated forms, profile editing, and logout confirmation
4eab849 Lab 3: add five independent StatefulShellRoute tabs
5c420fe Lab 3: define routes and ID-based navigation
6ffd49a Lab 3: add go_router dependency
c2b88af Add Lab 2: RunBaza interface and animated distance screens
```

Исходные материалы

L4_State_si_Repository_RU.pdf: задания 1-7 и критерии оценки.

github.com/HamuevOleg/PAM: проект, ветки лабораторных и история изменений.

pub.dev/packages/provider: Provider, watch, read и Consumer.

api.flutter.dev/flutter/foundation/ChangeNotifier-class.html: уведомления подписчиков.

dart.dev/language/class-modifiers#sealed: закрытая иерархия Result.

Визуальная основа: Samepace, main 438e942; существующая адаптация RunBaza из Л3. Анимации городов сохранены из Л2.